Midterm Project Report
Project Title: Comparative Analysis of Modern Tabular ML for Network Intrusion Detection
Team Members: Farman H. Sayem, Jace C. Rea, Kunle S. Oguntoye
Date: April 3rd, 2026

1. Project Overview

This project evaluates modern gradient boosting models for network intrusion detection within a reproducible machine learning framework. The study focuses on three tree-based tabular learning methods, LightGBM, XGBoost, and CatBoost, using the CSE-CIC-IDS2018 dataset as the primary benchmark and CIC-IDS2017 for cross-dataset drift analysis.

The core problem is that intrusion detection is a highly imbalanced classification task in which malicious traffic is rare compared with benign traffic. In this setting, overall accuracy can be misleading because a model may perform well on the majority class while still missing important attacks. A practically useful intrusion detection model must therefore be evaluated not only by classification performance, but also by its ability to support threshold selection, probability-based risk scoring, and robustness under changing traffic conditions.

The overall goal of this project is to build a reproducible evaluation framework that compares LightGBM, XGBoost, and CatBoost under class imbalance, evaluates probability calibration methods for improved risk estimation, and assesses how model performance changes under dataset drift between CIC-IDS2017 and CSE-CIC-IDS2018. This remains consistent with the original proposal, which emphasized calibrated performance and cross-dataset robustness rather than algorithmic novelty.

2. Motivation and Relevance

This project matters because intrusion detection remains a central cybersecurity challenge in environments with large volumes of network traffic and relatively rare malicious events. In such a setting, the cost of false negatives can be high because missed attacks may allow unauthorized access, persistence, or data exfiltration. At the same time, excessive false positives can overwhelm analysts and reduce the practical value of a detection system.

Many machine learning studies on intrusion detection report strong overall classification results, but fewer focus on deployment-relevant issues such as probability calibration, threshold selection, and robustness to dataset drift. These issues are especially important in operational systems because security analysts often rely on probability outputs to prioritize alerts and tune detection thresholds. A model that ranks attacks well but produces unreliable probabilities may still be difficult to deploy effectively.

By comparing three strong tabular boosting models, incorporating calibration analysis, and examining drift between related CIC intrusion datasets, this project addresses a practical cybersecurity problem with direct relevance to real-world detection systems.

3. Progress Completed So Far

At the midpoint of the semester, we have completed the implementation of a modular end-to-end pipeline in Google Colab for data preparation, preprocessing, model development, evaluation, and drift analysis.

For dataset preparation, we implemented a loading workflow for CSE-CIC-IDS2018 with stratified sampling to create a manageable subset while preserving the original class ratio. This was necessary because the full dataset is very large and impractical for repeated experimentation in Colab. The pipeline is also structured to support CIC-IDS2017 as a secondary dataset for later drift evaluation.

In the current implementation, we work with a 100,000-row subset of the 2018 dataset (15% attack rate) and a 50,000-row subset of the 2017 dataset (12% attack rate), ensuring consistency while remaining computationally efficient.

For preprocessing, we implemented a cleaning function tailored to CIC-style flow data. This function replaces infinite values with missing values, removes duplicate rows, drops non-generalizable identifier columns such as timestamp, source and destination IP addresses, ports, protocol, and flow ID when present, removes constant numeric columns, and clips extreme numeric outliers at the 99.9th percentile. Missing values are then handled through a preprocessing pipeline that is fit on the training set only and applied consistently to validation, test, and cross-dataset data in order to avoid leakage.

In our sampled datasets, no duplicate rows or constant columns were detected, and the final cleaned datasets retained 20 usable numerical features with effectively zero missing values after preprocessing.

We also completed exploratory data analysis, including class distribution inspection, selected feature distribution plots, and a correlation heatmap for a subset of features. This step helped verify the imbalance structure of the data and provided a basic understanding of the cleaned feature space before model training.

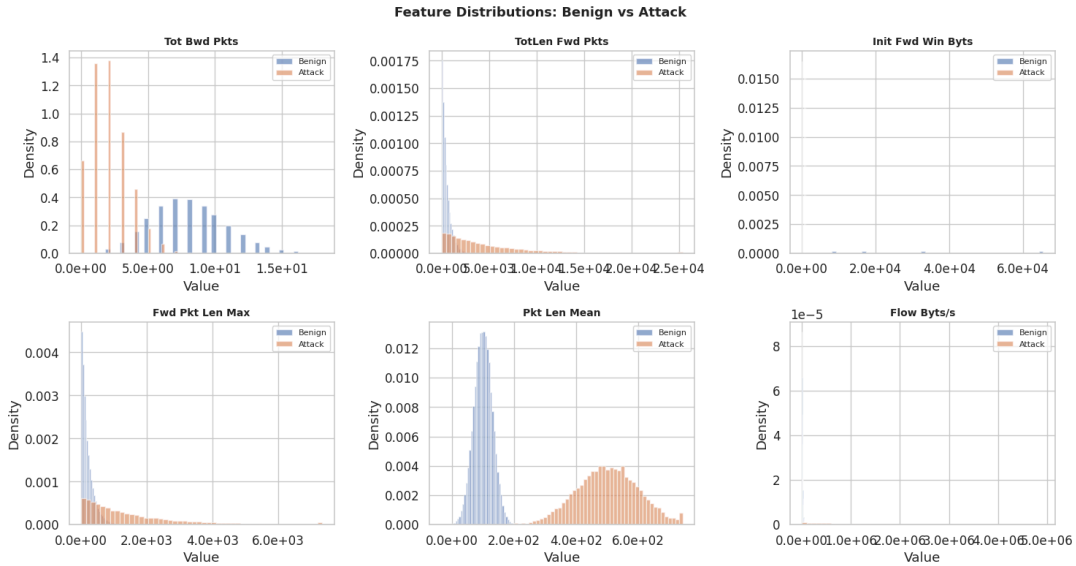

Figure 1. Selected Feature Distributions for Benign vs. Attack Traffic (CSE-CIC-IDS2018 subset).

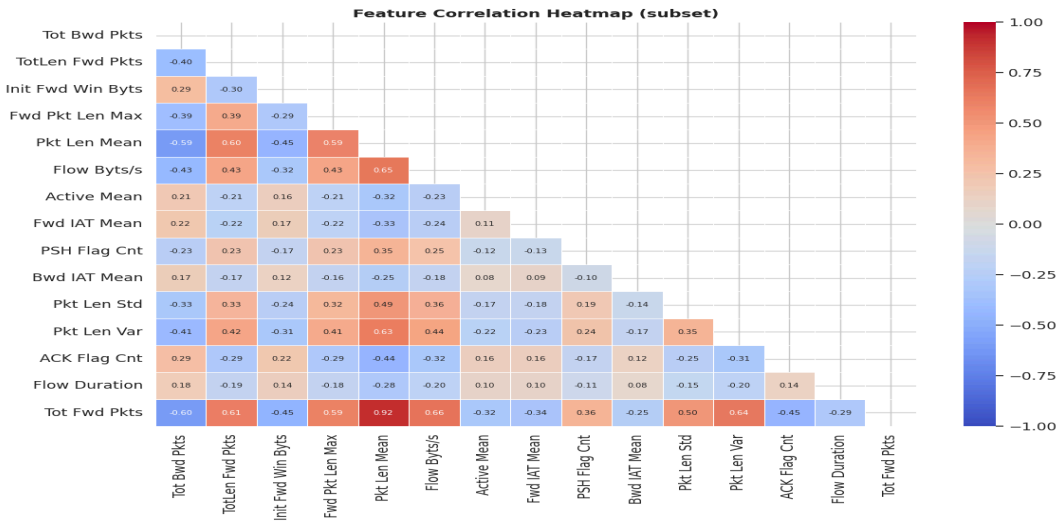

Figure 2. Feature Correlation Heatmap for Selected Numeric Features (CSE-CIC-IDS2018 subset).

The observed class imbalance is approximately 5.7:1 (benign to attack), confirming the need for imbalance-aware training strategies. For system design, we implemented a stratified 70/15/15 split for training, validation, and testing. The training split is used for learning and resampling, the validation split is reserved for calibration and threshold analysis, and the test split is held out for final evaluation. The notebook also preprocesses the 2017 dataset with the fitted 2018 pipeline when feature alignment allows, supporting a cleaner drift comparison.

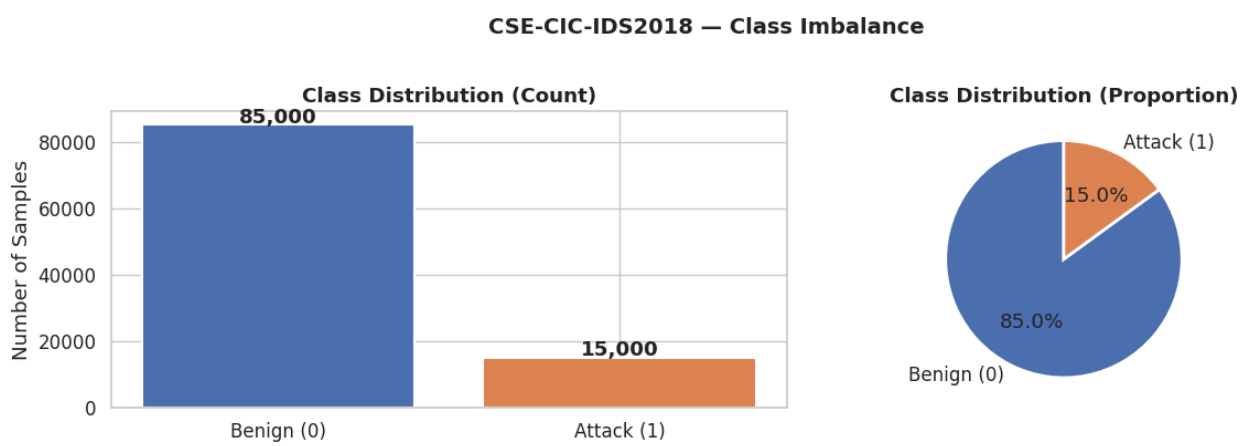

Figure 3. Attack vs. Benign Class Distribution (CSE-CIC-IDS2018 Subset)

The resulting splits maintain consistent attack ratios (~15%) across train (69,999 samples), validation (15,001 samples), and test (15,000 samples), confirming correct stratification. The aligned 2017 dataset (50,000 samples) preserves its original 12% attack rate for drift evaluation.

For class imbalance, we implemented both class weighting and SMOTE so the two strategies can be compared directly. Class weights are passed into the boosting models where appropriate, while SMOTE is applied only to the training data.

Computed class weights assign a significantly higher penalty to the minority class (attack), with a scale_pos_weight of approximately 5.67. SMOTE rebalancing increases the training set from 69,999 to 118,998 samples and produces a perfectly balanced 50% attack rate, enabling controlled comparison between weighting and oversampling strategies.

For model development, we implemented baseline versions of LightGBM, XGBoost, and CatBoost under comparable settings and recorded training time as a practical metric. We also implemented RandomizedSearchCV-based hyperparameter tuning using PR-AUC as the scoring objective, which is more meaningful than accuracy for imbalanced intrusion detection.

Initial training results show very fast training times, with LightGBM (~0.5s) and XGBoost (~0.6s) outperforming CatBoost (~1.0s) in speed, while all models successfully train under the unified pipeline. A SMOTE-based LightGBM variant was also trained efficiently (~0.4s), confirming feasibility of resampling approaches within the runtime constraints.

Finally, we built the complete evaluation framework for the second half of the project. This includes precision, recall, F1-score, ROC-AUC, PR-AUC, Brier score, false positive rate, confusion matrices, ROC and precision-recall curves, calibration analysis using Platt scaling and isotonic regression, threshold sweeps, KS-based drift analysis, cross-dataset performance comparison, and an aggregated results summary table.

4. Preliminary Results

The strongest preliminary result at this point is that the project has moved beyond the planning stage and now has a complete experimental framework implemented in code. The current notebook supports all three central project aims: comparative model evaluation, probability calibration, and drift analysis.

Early implementation has already highlighted several important characteristics of the problem. First, the CSE-CIC-IDS2018 dataset requires careful subsetting and preprocessing because of its size and the presence of data quality issues such as infinite values, duplicates, and non-generalizable identifiers. Second, evaluation for this task must go beyond accuracy and include threshold-sensitive measures such as recall and false positive rate, along with threshold-independent measures such as PR-AUC and calibration-sensitive measures such as Brier score. Third, the notebook structure makes it possible to compare models not only at the default 0.5 threshold, but also across a full threshold sweep, which is more realistic for operational intrusion detection. Additionally, preliminary system-level results confirm that the full pipeline, from data loading through preprocessing, stratified splitting, imbalance handling, and model training, executes successfully and efficiently on the sampled datasets, with all baseline models trained in under one second.

At this stage, the report is best interpreted as showing that the experimental infrastructure is complete and that the final comparative analysis is ready to be executed and documented. If final model outputs are

available before submission, this section should be updated with a concise summary table containing Recall, PR-AUC, ROC-AUC, Brier Score, and training time for LightGBM, XGBoost, and CatBoost. If those outputs are not yet finalized, it is more accurate to describe the notebook as providing a fully implemented framework for these comparisons rather than claiming fixed ranking results that are not yet shown.

5. Challenges and Limitations

One major challenge has been data scale. The full CSE-CIC-IDS2018 dataset is large enough to create memory and runtime issues in Google Colab, so a controlled stratified sampling process was necessary to keep the experiments reproducible and manageable.

A second challenge has been data cleaning. The CIC datasets include infinite values in certain flow statistics, duplicate rows, and columns that may not generalize well across environments. This required a custom cleaning function rather than a minimal off-the-shelf preprocessing approach.

A third challenge is cross-dataset consistency for drift analysis. Although CIC-IDS2017 and CSE-CIC-IDS2018 are related benchmark datasets, their features and formatting are not perfectly identical. Aligning them for meaningful drift analysis and cross-dataset evaluation requires additional care.

Another limitation is computational cost. Hyperparameter tuning, calibration, threshold analysis, and drift testing all increase runtime, so experimentation must be prioritized carefully within the semester timeline and available compute constraints.

6. Plan for the Rest of the Semester

The next step is to execute and document the full baseline comparison among LightGBM, XGBoost, and CatBoost on the held-out test set. We will make sure that all models are trained properly using the same stratified 70/15/15 train, validation, and test split, with no leakage between splits. We will report recall, precision, F1-score, ROC-AUC, PR-AUC, false positive rate, Brier score, and training time for each model under consistent evaluation conditions.

To better address class imbalance, we will systematically compare class weighting and SMOTE-based resampling. In particular, SMOTE will be applied strictly to the training set, while validation and test sets will remain unchanged to preserve real-world class distributions. We will analyze how each imbalance-handling strategy affects recall, PR-AUC, and false positive rate, especially for the minority (attack) class.

After baseline comparison, we will complete hyperparameter tuning using RandomizedSearchCV with PR-AUC as the primary objective. The tuning process will be performed using cross-validation within the training set only, and final model selection will be based on validation performance. This will allow us to determine whether tuned models materially improve minority-class detection performance without overfitting.

We will then finalize probability calibration using both Platt scaling and isotonic regression, applied on the validation set. We will evaluate calibration quality using reliability curves and Brier score, and compare calibrated vs. uncalibrated outputs. This step is important because the project is concerned not only with classification performance, but also with the quality of predicted probabilities for risk scoring and threshold selection.

Next, we will complete threshold optimization by performing a full threshold sweep on validation data and selecting operating points based on specific objectives, such as maximizing recall at a fixed false positive rate or balancing F1-score. The selected thresholds will then be evaluated on the test set to simulate realistic deployment conditions.

Finally, we will complete the drift analysis by aligning features between the 2018 and 2017 datasets and applying the preprocessing pipeline fitted on the 2018 training data. We will use Kolmogorov-Smirnov statistics to quantify feature-level distribution shifts and then evaluate how model performance degrades when applied to the 2017 dataset. This will provide a clearer measure of cross-dataset robustness and generalization.

References

Ahmad, Z., Khan, A. S., Shiang, C.-W., Abdullah, J., & Ahmad, F. (2021). *Network intrusion detection system: A systematic study of machine learning and deep learning approaches*. *Transactions on Emerging Telecommunications Technologies, 32*(1), e4150. https://doi.org/10.1002/ett.4150

Bouzaachane, K., El Guarmah, E. M., Alnajim, A. M., & Khan, S. (2025). *Addressing modern cybersecurity challenges: A hybrid machine learning and deep learning approach for network intrusion detection*. *Computers, Materials & Continua, 84*(2), 2391–2410. https://doi.org/10.32604/cmc.2025.065031

Gharib, A. H., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)* (pp. 108–116). https://doi.org/10.5220/0006639801080116

Leevy, J. L., & Khoshgoftaar, T. M. (2020). *A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 big data*. *Journal of Big Data*. https://doi.org/10.1186/s40537-020-00382-x

Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). *LightGBM: A highly efficient gradient boosting decision tree*. In *Advances in Neural Information Processing Systems, 30* https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html(pp. 3146–3154).